

CMPT 310 - Project Milestone 2

Project Recap

Our project is an AI agent that learns to play the game 2048 autonomously. The input is the current 4×4 game board, and the output is one of four possible actions (up, down, left, or right). We use reinforcement learning as our primary AI approach, integrated with an Expectimax search agent. A Random agent and the original heuristic-based Expectimax agent from Milestone 1 are used as baseline methods for comparison.

Accomplishments with Proof

Accomplishment 1: Implemented N-Tuple Network Value Function

To enable reinforcement learning, we replaced the manually designed heuristic evaluation function from our Expectimax agent with a learned value function. We implemented an N-tuple network that estimates the value of a given 2048 board state by learning patterns from board configurations. The network represents the 4×4 game board as multiple groups of tuples. Each tuple maps the observed tile values into lookup table entries, and the overall value of a board is calculated by summing the values from all tuple patterns. To improve generalization, we implemented symmetry handling by evaluating rotated and reflected versions of the board. The N-tuple network serves as the value function $V(s)$, where higher values represent board states that are expected to achieve higher future rewards. Initially, all lookup table values are initialized to zero, and they are gradually updated during training through experience generated from self-play.

Proof: code excerpts from `agents/n_tuple_network.py`

```
15 Coordinate = Tuple[int, int]
14 TuplePattern = Tuple[Coordinate, ...]
13 PathLike = Union[str, "Path"]
12
11
10 def _default_tuples() -> Tuple[TuplePattern, ...]:
9     rows = [tuple((row, col) for col in range(BOARD_SIZE)) for row in range(BOARD_SIZE)]
8     columns = [tuple((row, col) for row in range(BOARD_SIZE)) for col in range(BOARD_SIZE)]
7     squares = [
6         tuple((row + dr, col + dc) for dr in range(2) for dc in range(2))
5         for row in range(BOARD_SIZE - 1)
4         for col in range(BOARD_SIZE - 1)
3     ]
2     return tuple(rows + columns + squares)
1
38
1 DEFAULT_TUPLES = _default_tuples()
2
```

```

167 def feature_indices(self, grid: np.ndarray, validate: bool = True) -> np.ndarray:
1   """Return the table index selected by each symmetry and tuple.
2
3   Pass validate=False in hot loops once the caller guarantees a legal board.
4   """
5   board = np.asarray(grid)
6   if board.shape != (BOARD_SIZE, BOARD_SIZE):
7       raise ValueError("grid must have shape (4, 4)")
8
9   ranks = self._ranks_checked(board) if validate else _RANK_LUT[board]
10
11   selected = ranks.reshape(-1)[self._symmetry_cells].astype(np.int64, copy=False)
12   return selected @ self._place_values
13
14 def value(self, grid: np.ndarray, validate: bool = True) -> float:
15   indices = self.feature_indices(grid, validate=validate)
16   table_numbers = np.arange(len(self.tuples), dtype=np.intp)[None, :]
17   return float(self.tables[table_numbers, indices].sum(dtype=np.float64))
18

```

```

12
13 def update(self, grid: np.ndarray, delta: float) -> None:
14   """Distribute delta evenly over every table lookup used by value()."""
15   try:
16       numeric_delta = float(delta)
17   except (TypeError, ValueError) as error:
18       raise TypeError("delta must be a finite real number") from error
19   if not np.isfinite(numeric_delta):
20       raise ValueError("delta must be finite")
21
22   indices = self.feature_indices(grid, validate=False)
23   amount_per_lookup = numeric_delta / indices.size
24   offsets = np.arange(len(self.tuples), dtype=np.int64) * TABLE_SIZE
25   np.add.at(self.tables.reshape(-1), (indices + offsets).reshape(-1), amount_per_lookup)
26

```

The excerpts show the 17 tuple patterns extracted from the board (4 rows, 4 columns, 9 overlapping 2×2 squares), the conversion of a board into lookup table indices across all 8 symmetries, and the paired `value()` and `update()` methods.

This replaces the hand-designed evaluation function from Milestone 1 with one that is learned. The symmetry handling is what makes the learning practical: the eight rotations and reflections of a board share the same lookup tables, so one observed position writes into $8 \times 17 = 136$ table entries at once and the agent generalizes from far fewer games. The `value()` and `update()` pair is the interface TD learning needs, which is what let the training pipeline in Accomplishment 2 be built on top of it.

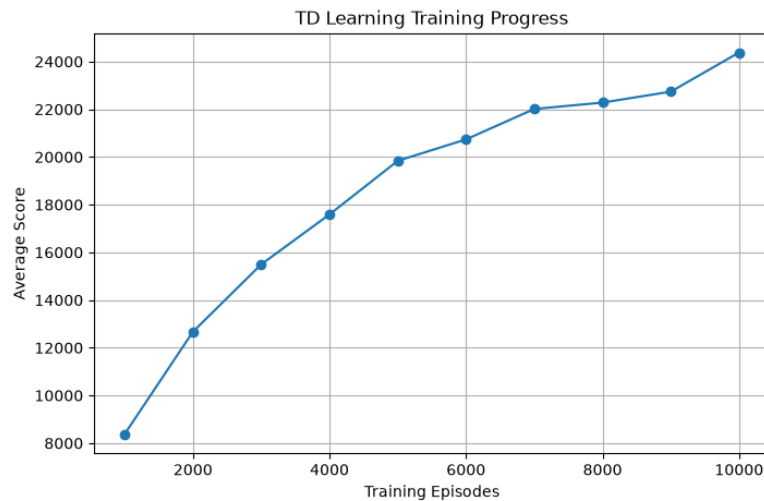
Accomplishment 2: Implemented TD Learning Self-Play Training Pipeline

We implemented a Temporal Difference (TD) learning training pipeline that allows the agent to improve its decision-making through self-play. The agent generates its own experience by repeatedly playing complete 2048 games. During each episode, the agent observes the current board state, selects the move with the highest estimated value using the current N-tuple network, performs the action, and receives a reward based on the score gained from tile merges. The TD error is calculated by comparing the current value estimate with the observed reward and the estimated future value. Then, the N-tuple

network is updated using this error to gradually improve its prediction. We also implemented checkpoint saving that saves learned values periodically (every 1000 episodes). This helps us track progress and enables future evaluation or continued training.

The concrete settings behind the submitted agent are as follows. Each board cell is encoded as the exponent of its tile value (0 for an empty cell, k for a tile of 2^k), which is what makes each four-cell tuple a base-16 lookup index. The reward for a move is the sum of the values of the tiles created by merges on that move. The submitted value function was trained for 10,000 self-play episodes with a learning rate of $\alpha = 0.05$, applied to the afterstate reached after the player moves but before the random tile spawns.

Proof: training progress graph and the TD update in `training/td_learning.py`



```

26     best_action = None
25     best_afterstate = None
24     best_reward = 0.0
23     best_value = float("-inf")
22
21     for action, move in MOVES.items():
20         afterstate, merged_values, changed = move(state)
19         if not changed:
18             continue
17
16         reward = points_after_merge(merged_values)
15         value = reward + self.network.value(afterstate, validate=False)
14
13         if value > best_value:
12             best_action = action
11             best_afterstate = afterstate
10             best_reward = reward
9             best_value = value
8
7     return best_action, best_afterstate, best_reward, best_value
6
5     def play_episode(self):
4         """One self-play game with TD(0) updates over afterstates.
3
2         V(afterstate) estimates FUTURE reward only, so the target for afterstate A1 is
1         R2 + V(A2), where R2 is the reward of the move taken from the NEXT board.
133     The current move's own reward is used for action selection, not for the target.
1         """

```

The graph plots the average score over each block of 1000 training episodes, rising from roughly 8,400 after the first 1000 episodes to about 24,400 at 10,000. The code excerpt shows the TD(0) update: the target for an afterstate is the reward of the following move plus the estimated value of the next afterstate, and the network is corrected by that error scaled by the learning rate.

The value function starts at zero everywhere, so all of this improvement comes from experience the agent generated itself. The curve rising steadily and not collapsing is evidence the TD updates are stable, which is the main risk with a learning rate applied to a lookup table.

Accomplishment 3: Integrated RL Agent with Existing 2048 System and Completed Interactive UI

We integrated the reinforcement learning components with our existing 2048 environment, which includes board movement, tile merging, random tile spawning, scoring, and game termination logic. The reinforcement learning pipeline was designed to interact directly with this environment, allowing the N-tuple network value function to learn board evaluations through self-play without modifying the core game mechanics. Additionally, we completed the graphical user interface using Pygame. It includes a main menu where users can select between the agents, a 4×4 game board, score and best score display, and tile animations for movement and merging.

Proof: the evaluation function that lets one search use either value function, and a table

comparing the previous Expectimax agent, the Expectimax agent using the learned value function, and the Random baseline

```
8
7     # Learned value function or the previous heuristic can be used.
6     def evaluate(self, state):
5         if self.network is not None:
4             return self.network.value(state, validate=False)
3         empty = len(empty_cells(state))
2         max_tile = np.max(state)
1         return (100 * empty) + max_tile
98
```

Agent	Games	Avg Score	Median	Best	Best Tile	Reached 2048
Random	15	992	720	2,772	256	0/15
Expectimax (heuristic)	15	9,974	7,184	26,768	2048	1/15
Expectimax + RL	15	48,487	37,880	78,224	4096	15/15

All three agents played 15 games on the same environment, with the RL agent using the trained value function inside the same Expectimax search at depth 2.

The learned value function outperforms the hand-written heuristic by roughly $4.9\times$ and reaches the 2048 tile in every game, where the heuristic reaches it once in 15 and the random baseline never does. The comparison also isolates what the search contributes: the training graph shows the value function alone averaging about 24,400 when picking moves greedily, and placing that same network inside the depth-2 search roughly doubles it. Both components are needed, which is why the heuristic Expectimax is kept as a baseline rather than replaced.

Proof for UI: A video demonstration showing the complete UI flow, including the main menu, agent selection, gameplay, tile animations, and score updates.

Video Link: [UI Demonstration Video](#)

Challenges

The main challenge we encountered during Milestone 2 was designing the reinforcement learning component of our agent. A major difficulty was selecting an appropriate value representation and training approach for the 2048 environment. We needed to consider factors such as the stochastic nature of tile spawning, the large number of possible board configurations, and the computational cost of training. We explored different reinforcement learning approaches and evaluated how each method would integrate with our existing game environment.

Another challenge was designing a training process that allowed the agent to improve re-

liably. We needed to determine how the agent should generate experience, how rewards should influence learning, and how value updates should be applied without causing unstable learning. Through research and experimentation, we developed a self-play training pipeline using TD learning and an N-tuple network, which allowed the agent to efficiently learn board evaluation patterns from its own gameplay experience.

Changes from Original Plan

Our overall project scope and objective remained unchanged. However, two implementation decisions changed from our original proposal. Initially, we planned to implement a neural-network-based Q-learning agent using PyTorch. After further research into reinforcement learning approaches for 2048, we decided to replace the planned neural network value representation with an N-tuple network and replace Q-learning with a Temporal Difference (TD) learning algorithm.

This change was made because N-tuple networks are well-suited for the 2048 environment, allowing the agent to efficiently learn board evaluation patterns through self-play while requiring significantly less computational overhead than training a neural network. Additionally, TD learning is a better fit for our implementation because 2048 has deterministic player actions followed by stochastic tile spawning. Instead of learning separate values for each possible action, TD learning allows the agent to directly learn the value of the resulting board position after each move and update its estimate based on future rewards.

These adjustments maintained the original goal of developing a reinforcement learning agent while providing a more practical and efficient approach that could be completed within the project timeline.

GitHub repo link: <https://github.com/praneetkb/2048-Master>